



FedBook-Sem

A social bookstore that talks to other social bookstores

What is FedBook-Sem?

- An online bookstore with a social layer — reviews, annotations, followers.
- But unlike Goodreads or Storygraph, it is federated — instances can talk to each other and to Mastodon.
- It also learns from the wider web — reading what people say about books on Reddit, YouTube, Bluesky, and Mastodon — to make better recommendations.

Why It Matters

Today

- Your reviews live inside one product.
- Your followers cannot leave with you.
- Recommendations use only the platform's own data.
- Annotations are locked to one app.

With FedBook-Sem

- Your identity is a portable ActivityPub actor.
- Followers reach you across servers.
- Recommendations blend semantic, social, and cross-platform signal.
- Annotations follow a W3C standard used in academic research.

What You Can Do

- Browse a curated book catalogue with covers, subjects, and star ratings.
- Write reviews, like and reshare reviews from other users.
- Annotate passages inside the book reader — highlight, comment, or tag.
- Follow other readers and see their activity in a personalised feed.
- Get recommendations that also consider what people are saying about a book on social media.

How the Pieces Fit

React app	The interface you use in your browser.
Express API	The backend that handles reviews, follows, and federation.
Neo4j graph	Stores the social network — who follows whom, what they read.
ML service	Ranks and recommends books using several models.
Streamlit dashboard	A developer view that shows how the recommender scored each book.
Ingestion worker	Pulls book chatter from Reddit, YouTube, Bluesky, Mastodon (privately).

The Architecture in Layers

What you see	The React app in your browser and the Streamlit dashboard for developers.
What handles requests	Two servers — a Node.js one for the social features, a Python one for the recommender.
Where knowledge lives	A Neo4j graph database — think of it as a giant map of people, books, and their connections.
Supporting services	Redis for fast caching, PostgreSQL as a backup relational store.
What it talks to outside	Other ActivityPub servers (Mastodon-style), plus Reddit / YouTube / Bluesky / Mastodon for reception signal.

What Happens When You Follow Someone

- You click Follow on a user profile.
- Your server looks up the other person via WebFinger (like an email lookup for social identities).
- Your server sends a cryptographically signed 'Follow' message to their server's inbox.
- Their server checks the signature is really yours, records the follow, replies with an 'Accept'.
- You now see each other's activity — even if you're on different servers.

What Happens When You Open a Book Page

- The React app asks the ML service for similar books.
- ML converts the book into a mathematical fingerprint (an embedding vector).
- It finds the 50 most similar fingerprints already in the database.
- Re-scores them using: what people are saying about them online, who else read them, how fresh the chatter is.
- Returns the top 10 with a mini scorecard for each — sentiment per platform, star rating, subjects.

Where Different Kinds of Data Live

Neo4j graph database	People, books, follows, reviews, annotations, per-platform reception counts.
Redis	Fast temporary data — session cache, background job queues.
PostgreSQL	Reserved for future relational data (e.g. purchase history).
Disk (inside the ML container)	Downloaded AI models — about 1 GB total, cached so no internet is needed at startup.
Nowhere	The actual text of anyone's Reddit / YouTube / Bluesky / Mastodon posts — deliberately not stored.

How Recommendations Work — In Plain Terms

- Semantic similarity — 'books that read like this one'.
- Collaborative filtering — 'readers like you also liked'.
- Cross-platform reception — 'what is the internet saying about this book right now?'.
- A learned model blends these signals into a single ranked list.

Privacy by Design

- The system never stores the actual text of anyone's Reddit / YouTube / Bluesky / Mastodon posts.
- It only keeps counts of how many mentions were positive, neutral, or negative.
- YouTube data is automatically deleted after 30 days.
- Reddit deletions are honoured — if the original post is gone, our count is scrubbed.

Built On Open Standards

- ActivityPub — the same protocol Mastodon uses. Any ActivityPub server can talk to FedBook-Sem.
- W3C Web Annotation — the same standard used in scholarly text and hypothes.is.
- WebFinger — lets you find users as acct:name@domain, just like email.
- HTTP Signatures — every message between servers is cryptographically signed.

Under the Hood

Backend	Node.js + Express + Neo4j
Frontend	React + custom design
ML	Python + FastAPI + Hugging Face + LightFM
Dashboard	Streamlit
Infrastructure	Docker Compose (Neo4j, Redis, PostgreSQL)

How It Runs

- `docker compose up -d` brings up the databases and the ML services.
- `npm run dev:backend` and `npm run dev:frontend` start the app locally.
- Sign in as one of the demo users (alice / bob / carol) and start reading, reviewing, annotating.
- The dashboard at `:8501` lets you inspect the recommender's decisions.

What's Next

- User study with real readers (task-completion, subjective satisfaction).
- Federation interop tests with Mastodon and BookWorm.
- Faster reranker to trim /recommend/similar latency.
- Onboarding taste-prompt to help brand-new users get good recommendations.

Thank You

Questions?

— SLIIT R26-IT-107 · IT22922670